

Product Requirements Document

Online Marketplace Order Platform

1. Product Summary

The Online Marketplace Order Platform enables buyers to purchase products from third-party sellers through a centralized marketplace experience. The platform manages checkout, order creation, payment authorization, seller fulfillment coordination, shipment status, cancellations, refunds, and customer support visibility.

The product is intended for marketplaces where inventory, fulfillment, and seller operations are distributed across multiple vendors, while the marketplace operator owns the buyer checkout experience and order lifecycle coordination.

2. Product Goals

The product should:

1. Allow buyers to place orders containing products from one or more sellers.
2. Coordinate payment authorization and capture.
3. Split marketplace orders into seller-specific fulfillment orders.
4. Provide sellers with clear fulfillment responsibilities.
5. Track order, payment, shipment, cancellation, and refund status.
6. Give customer support teams a unified view of the order lifecycle.
7. Maintain auditability across buyer, seller, payment, and support actions.
8. Support reliable integration with external payment, tax, shipping, seller, and notification services.

3. Non-Goals

The product will not:

1. Provide a full product information management system.
2. Replace seller inventory management systems.
3. Operate warehouse robotics, carrier systems, or physical fulfillment infrastructure.
4. Provide accounting, tax filing, or seller payout reconciliation in the initial release.
5. Provide marketing campaign management.
6. Support auction-style purchasing in the initial release.
7. Support peer-to-peer negotiation or custom quote workflows in the initial release.
8. Store raw payment card data.

4. Target Users

User	Description
Buyer	Browses products, checks out, pays, tracks orders, and requests cancellation or refund.
Seller	Receives fulfillment orders, confirms availability, ships products, and updates fulfillment status.
Customer Support Agent	Reviews orders, assists buyers, initiates approved cancellations or refunds, and resolves order issues.
Marketplace Administrator	Configures marketplace rules, seller status, order policies, and operational settings.
Operations Manager	Monitors order flow, seller response, fulfillment exceptions, and unresolved integration failures.

5. External Systems

External System	Description
Payment Gateway	Authorizes, captures, voids, and refunds payments.
Tax Calculation Service	Calculates sales tax, VAT, or other applicable transaction taxes.
Shipping Carrier Service	Provides shipping rates, labels, tracking events, and delivery status where supported.
Notification Provider	Sends email, SMS, or push notifications for order lifecycle events.
Seller System	Optional seller-side system for inventory confirmation, fulfillment updates, or order import.
Fraud Screening Service	Optional service for risk scoring and fraud review.
Identity Provider	Optional system used to authenticate sellers, support agents, and administrators.

6. Primary User Problems

Buyers need a simple and trustworthy checkout experience even when an order involves multiple sellers. Sellers need clear, actionable fulfillment instructions without taking responsibility for the entire marketplace order. Customer support teams need a unified

order history so they can resolve disputes, failed payments, shipment delays, cancellations, and refunds.

The marketplace operator needs a reliable order coordination layer that can handle distributed fulfillment, payment state, external service failures, and audit requirements without creating inconsistent order states.

7. Product Capabilities

7.1 Cart and Checkout

Buyers should be able to:

1. Add available products to a cart.
2. Review item price, quantity, seller, shipping option, estimated tax, and total cost.
3. Enter or select a shipping address.
4. Select a supported payment method.
5. Review marketplace terms before purchase.
6. Submit the order for processing.
7. Receive confirmation when the order is accepted.

The checkout experience should prevent buyers from placing orders with unavailable items, invalid addresses, unsupported shipping destinations, or failed payment authorization.

7.2 Order Creation

The platform should create a marketplace order after successful checkout validation.

The system should:

1. Assign a unique marketplace order ID.
2. Preserve the buyer-visible order summary.
3. Store the finalized checkout total.
4. Store the product, seller, price, tax, shipping, and discount details accepted by the buyer.
5. Track order status.
6. Track item-level status.
7. Track payment status.
8. Track fulfillment status.

Order history should preserve what the buyer saw and accepted at the time of purchase, even if product, seller, tax, or shipping information changes later.

7.3 Seller Order Splitting

If a marketplace order contains products from multiple sellers, the platform should split it into seller-specific fulfillment orders.

The system should:

1. Assign one or more seller order IDs.
2. Preserve the relationship between marketplace order and seller fulfillment orders.
3. Send each seller only the fulfillment information relevant to that seller.
4. Track seller-specific acceptance, rejection, shipment, and exception status.
5. Support item-level status where cancellation, shipment, or refund applies to only part of an order.

7.4 Payment Authorization and Capture

The system should coordinate payment operations through an external payment gateway.

The system should support:

1. Payment authorization.
2. Payment capture.
3. Payment void when an order cannot be accepted.
4. Full refund.
5. Partial refund where supported.
6. Recording payment status and gateway references.
7. Handling payment failures.
8. Safe retry of payment-related operations.

The platform should not mark an order as accepted until payment authorization and required order validations have completed successfully.

7.5 Tax and Pricing Finalization

The system should calculate final order totals before order submission.

The system should support:

1. Item subtotal calculation.
2. Shipping charge calculation or selection.
3. Tax calculation through an external tax service where configured.
4. Discounts or promotions where supported by marketplace policy.
5. Buyer-visible total before payment authorization.
6. Storage of the finalized order total used for payment authorization and audit.

7.6 Inventory and Availability Confirmation

The platform should verify product availability before completing checkout.

Availability confirmation may be based on:

1. Marketplace-held inventory snapshots.
2. Seller system integration.
3. Seller-provided availability feeds.
4. Manual seller confirmation after order placement.
5. Reservation windows.

The system should clearly distinguish between confirmed, pending, unavailable, backordered, and canceled item states.

7.7 Seller Fulfillment Coordination

Sellers should be able to:

1. View assigned seller orders.
2. Accept or reject fulfillment responsibility according to marketplace rules.
3. Confirm item availability.
4. Add shipment details.
5. Update fulfillment status.
6. Provide tracking information.
7. Communicate fulfillment exceptions to the marketplace.

The platform should notify sellers of new fulfillment orders and track fulfillment progress.

7.8 Shipment Tracking

The system should support shipment tracking for buyer and support visibility.

The system should:

1. Store shipment carrier, tracking number, and shipment status.
2. Receive or import carrier tracking events where supported.
3. Display buyer-facing shipment progress.
4. Notify buyers of shipment milestones.
5. Allow support agents to view shipment history.
6. Support multiple shipments per marketplace order.

7.9 Cancellation Management

The system should support cancellation workflows before fulfillment reaches a configured point of no return.

Cancellation may be initiated by:

1. Buyer.
2. Seller.

3. Customer Support Agent.
4. Marketplace operational policy.

The system should:

1. Validate whether cancellation is allowed.
2. Cancel affected items or seller orders.
3. Void or refund payment as appropriate.
4. Notify affected parties.
5. Record cancellation reason and actor.
6. Preserve a clear audit record of the cancellation decision.

7.10 Refund and Return Support

The system should support full and partial refunds.

Refunds may be triggered by:

1. Buyer support request.
2. Seller inability to fulfill.
3. Shipment failure.
4. Return approval.
5. Customer support resolution.
6. Marketplace policy adjustment.

The system should:

1. Calculate refundable amount.
2. Submit refund request to the payment gateway.
3. Record gateway response.
4. Update order and item status.
5. Notify buyer and seller where appropriate.
6. Maintain audit history.

Return logistics may be handled outside the MVP unless explicitly included in release scope.

7.11 Customer Support Order View

Customer Support Agents should be able to:

1. Search for orders by buyer, order ID, seller, item, payment reference, or shipment reference.
2. View marketplace order status.
3. View seller order status.
4. View payment history.

5. View shipment history.
6. View cancellation and refund history.
7. Initiate authorized support actions.
8. Add support notes.
9. See external integration failures that may affect the order.

The support view should help agents understand the complete order lifecycle without needing direct access to external payment, seller, or carrier systems.

7.12 Notifications

The platform should notify relevant users about major order events.

Notification events may include:

1. Order confirmation.
2. Payment failure.
3. Seller fulfillment request.
4. Seller order accepted or rejected.
5. Shipment created.
6. Shipment delivered.
7. Cancellation requested.
8. Cancellation completed.
9. Refund issued.
10. Support resolution.

Notifications should avoid exposing sensitive payment details.

8. Representative User Journeys

8.1 Buyer Places a Single-Seller Order

1. Buyer adds an item to cart.
2. Buyer enters shipping and payment details.
3. System calculates tax, shipping, and order total.
4. System verifies item availability.
5. System authorizes payment.
6. System creates marketplace order and seller order.
7. System notifies buyer and seller.
8. Seller fulfills the order.
9. System records shipment tracking.
10. Buyer receives shipment updates.

8.2 Buyer Places a Multi-Seller Order

1. Buyer adds items from multiple sellers to cart.
2. System displays item, seller, shipping, tax, and total cost.

3. Buyer submits checkout.
4. System validates availability and authorizes payment.
5. System creates one marketplace order.
6. System splits the order into seller-specific fulfillment orders.
7. Each seller receives only their assigned fulfillment order.
8. Buyer sees a unified order with multiple fulfillment sections.
9. Customer support can view marketplace-level and seller-level order status.

8.3 Seller Cannot Fulfill an Item

1. Seller receives a fulfillment order.
2. Seller reports that an item is unavailable.
3. System updates the affected item status.
4. System determines whether the remaining order can continue.
5. System initiates partial cancellation or support review.
6. System voids or refunds the affected amount as appropriate.
7. Buyer receives a status update.
8. System records seller exception and payment action.

8.4 Customer Support Issues a Refund

1. Buyer contacts customer support about a damaged item.
2. Support Agent searches for the order.
3. Support Agent reviews payment, shipment, seller, and item history.
4. Support Agent determines refund eligibility.
5. Support Agent initiates partial refund.
6. System submits refund request to payment gateway.
7. Payment gateway returns success or failure.
8. System updates order status.
9. System notifies buyer.
10. System records support action.

8.5 Payment Authorization Fails

1. Buyer submits checkout.
2. System validates cart and calculates final total.
3. System submits authorization request to payment gateway.
4. Payment gateway declines or fails the authorization.
5. System does not create an accepted order.
6. Buyer receives a clear payment failure message.
7. System records the failed checkout attempt for troubleshooting and fraud analysis where appropriate.

9. Functional Requirements

ID	Requirement
----	-------------

ID	Requirement
FR-001	The system shall allow buyers to submit checkout for one or more cart items.
FR-002	The system shall calculate final order totals before payment authorization.
FR-003	The system shall integrate with an external tax calculation service where configured.
FR-004	The system shall verify item availability before accepting an order.
FR-005	The system shall authorize payment through an external payment gateway.
FR-006	The system shall create a marketplace order after successful checkout validation.
FR-007	The system shall split marketplace orders into seller-specific fulfillment orders.
FR-008	The system shall track status at marketplace order, seller order, and item levels.
FR-009	The system shall notify sellers of new fulfillment orders.
FR-010	The system shall allow sellers to accept, reject, or update fulfillment orders.
FR-011	The system shall store shipment tracking information.
FR-012	The system shall update buyer-visible shipment status.
FR-013	The system shall support buyer, seller, support, and policy-driven cancellations.
FR-014	The system shall validate cancellation eligibility before canceling an order or item.
FR-015	The system shall support full refunds through the payment gateway.
FR-016	The system shall support partial refunds where enabled.
FR-017	The system shall record payment authorization, capture, void, and refund status.
FR-018	The system shall provide a support-facing order search and order detail view.

ID	Requirement
FR-019	The system shall allow authorized support agents to initiate approved cancellation or refund actions.
FR-020	The system shall send notifications for major order lifecycle events.
FR-021	The system shall maintain an audit trail of checkout, order, payment, fulfillment, cancellation, refund, and support actions.
FR-022	The system shall expose integration failure status for payment, tax, shipping, notification, fraud, and seller system integrations.
FR-023	The system shall support safe retry handling for checkout, payment, fulfillment, shipment, cancellation, and refund operations.
FR-024	The system shall preserve the buyer-accepted order details for historical review.

10. Non-Functional Requirements

Category	Requirement
Reliability	The system shall prevent inconsistent order and payment states during external integration failures.
Retry Safety	Order creation, payment operations, seller updates, shipment events, cancellations, and refunds shall be safe to retry.
Auditability	The system shall record significant lifecycle changes and the actor or system responsible for each change.
Security	Buyer, seller, support, and administrator access shall be authenticated and authorized by role.
Privacy	Sensitive buyer, payment, address, and seller information shall be protected and only exposed to authorized participants.
Payment Safety	The system shall not store raw payment credentials and shall use payment-token mechanisms provided by approved

Category	Requirement
	payment services.
Scalability	The system should support increased order volume, seller count, item count, and shipment event volume.
Performance	Checkout operations should complete quickly under normal traffic and degrade gracefully when external services are slow.
Availability	Buyer checkout and order status capabilities should remain highly available during marketplace operating hours.
Observability	The system shall provide operational visibility into order failures, payment failures, integration errors, and processing backlogs.
Consistency	Buyer-visible order state shall remain consistent with payment, seller fulfillment, cancellation, refund, and shipment state.
Configurability	Marketplace rules for cancellation, refund, seller acceptance, shipping options, notifications, and support actions should be configurable.

11. Data Requirements

The system should manage the following data:

1. Buyer order identity and contact references.
2. Marketplace order records.
3. Seller fulfillment order records.
4. Order item records.
5. Seller references.
6. Product and availability snapshots used during checkout.
7. Finalized price, tax, shipping, discount, and total records.
8. Payment authorization, capture, void, and refund records.
9. Shipment and tracking records.
10. Cancellation requests and outcomes.
11. Refund requests and outcomes.
12. Notification records.
13. Seller fulfillment status history.
14. Support notes and case references.

15. Integration request and response status.
16. Audit events.
17. Marketplace policy configuration.
18. Fraud screening status where configured.

Order records should preserve enough historical detail to explain what the buyer saw and accepted at checkout, even if product, seller, tax, or shipping configuration changes later.

12. Reporting and Operational Visibility

Marketplace operators and support teams should be able to view:

1. Order volume.
2. Checkout failure rate.
3. Payment authorization success and failure rates.
4. Seller fulfillment response times.
5. Unaccepted seller orders.
6. Orders blocked by integration failures.
7. Shipment delay indicators.
8. Cancellation and refund volume.
9. Support-driven order actions.
10. Multi-seller order completion rate.
11. Processing backlogs.
12. Unresolved external service failures.

13. Success Metrics

Metric	Target Direction
Checkout completion rate	Increase
Order creation failure rate	Decrease
Payment authorization failure due to platform errors	Decrease
Orders with inconsistent payment and order state	Near zero
Seller fulfillment response time	Decrease
Shipment tracking coverage	Increase
Buyer order-status support contacts	Decrease
Refund processing time	Decrease
Integration failures unresolved after one business day	Decrease
Support agent time to understand order history	Decrease
Multi-seller order success rate	Increase

14. Risks and Open Questions

Area	Question or Risk
Payment timing	Should payment be captured at order creation, seller acceptance, shipment, or delivery?
Seller inventory	Is inventory authoritative in the marketplace, seller systems, or both?
Multi-seller orders	Should buyers be allowed to cancel one seller portion without canceling the full marketplace order?
Tax handling	Which jurisdictions, tax types, and tax exemption scenarios must be supported?
Refund policy	Who can approve refunds, and what limits apply by role?
Seller SLA	What happens when sellers do not respond to fulfillment requests within the required window?
Shipment ownership	Are sellers responsible for buying labels, or does the marketplace generate carrier labels?
Fraud review	Should risky orders be held before seller fulfillment begins?
Customer support authority	Which support actions are allowed without seller approval?
Data retention	How long must order, payment, shipment, support, and audit records be retained?
Marketplace policy	Which rules are global, seller-specific, category-specific, or region-specific?

15. Release Scope

MVP

The MVP should include:

1. Buyer checkout submission.
2. Single-seller and multi-seller order creation.
3. Payment authorization through external payment gateway.
4. Basic tax calculation integration.
5. Seller fulfillment order creation.
6. Seller portal view for assigned orders.

7. Manual seller fulfillment status updates.
8. Buyer-visible order status.
9. Basic shipment tracking fields.
10. Email order notifications.
11. Support order search and order detail view.
12. Full order cancellation before shipment.
13. Full refund through payment gateway.
14. Audit logging for order, payment, seller, and support actions.

Post-MVP

Future releases may include:

1. Partial item cancellation.
2. Partial refunds.
3. Automated seller inventory reservation.
4. Carrier label generation.
5. Real-time carrier tracking webhooks.
6. Fraud screening integration.
7. Return merchandise authorization workflow.
8. Seller SLA automation.
9. Buyer self-service cancellation.
10. Buyer self-service return requests.
11. Marketplace policy engine.
12. Advanced support case integration.
13. Seller system API integration.
14. Multi-currency and cross-border tax support.
15. Seller payout and reconciliation integration.